

Compiladores

Aula 18: Otimização local e análise de fluxo de dados

Prof. Alex Torquato Souza Carneiro
alex.carneiro@impacta.edu.br

Agenda

- Blocos básicos
- Transformações locais
- Análises forward e backward

Blocos básicos

- A IR explicita operações e controle; as transformações preservam o comportamento observável.
- Através dos comandos básicos que o compilador entende, seremos capazes de representar sequências de instruções complexas.
- Uma etapa importante na compilação é a otimização do código gerado.

Blocos básicos

- A otimização tem como objetivo reduzir custo sem mudar resultados definidos, efeitos observáveis ou erros previstos.
- Cuidados semânticos:
  - Respeitar tipos e conversões de MINIC.
  - Preservar curto-circuito de && e ||.
  - Não apagar chamadas, read/print ou falhas observáveis.

Exemplo: multiplicar por zero?

```
x = f() * 0;
```

Reduzir para 0 pode remover a chamada f() e seus efeitos. A regra precisa refletir a semântica efetiva.

Blocos básicos

- Dentro do bloco básico, o fluxo é sequencial; desvio somente na última instrução.

```
1   a = 1
2   if c goto 5
3   b = a + 2
4   goto 6
5   b = a + 3
6   print(b)
7   return 0
```

Líderes

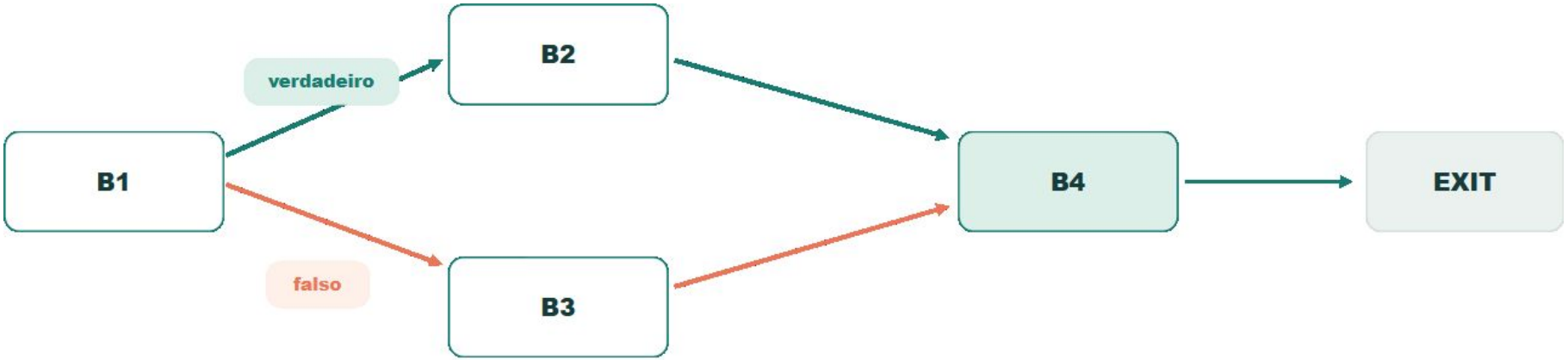
- Primeira instrução do procedimento.
- Destino de qualquer salto.
- Instrução logo após qualquer salto.

Partição resultante

B1 {1–2}	B2 {3–4}	B3 {5}	B4 {6–7}

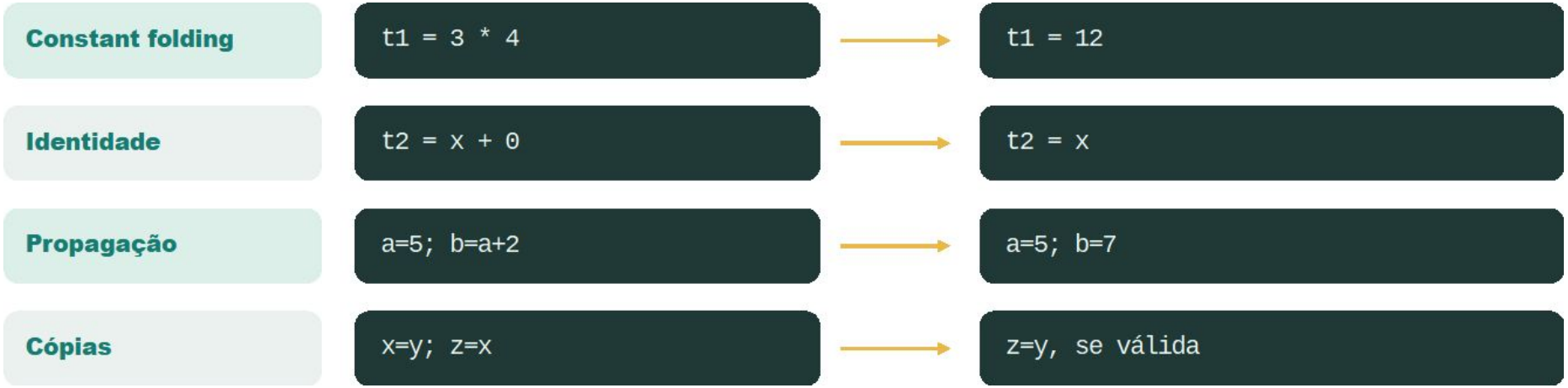
Blocos básicos

- A lógica do bloco básico é modelada em um grafo de fluxo, o qual registra caminhos possíveis, não execuções obrigatórias.
- Arestas registram transferências possíveis.



Transformações locais

- As transformações locais ocorrem dentro de um bloco.
- Cada substituição depende de tipos válidos e semântica compatível.



Divisor constante zero é erro a rejeitar — não uma simplificação.

Transformações locais

- As transformações devem compartilhar cálculos e remover só o que é dispensável.

Subexpressão comum

```
q = a + b;
r = a + b;
print(r);
```

Se a e b não mudaram, r pode reutilizar q. Remova q apenas se não houver uso.

Efeitos em vetores

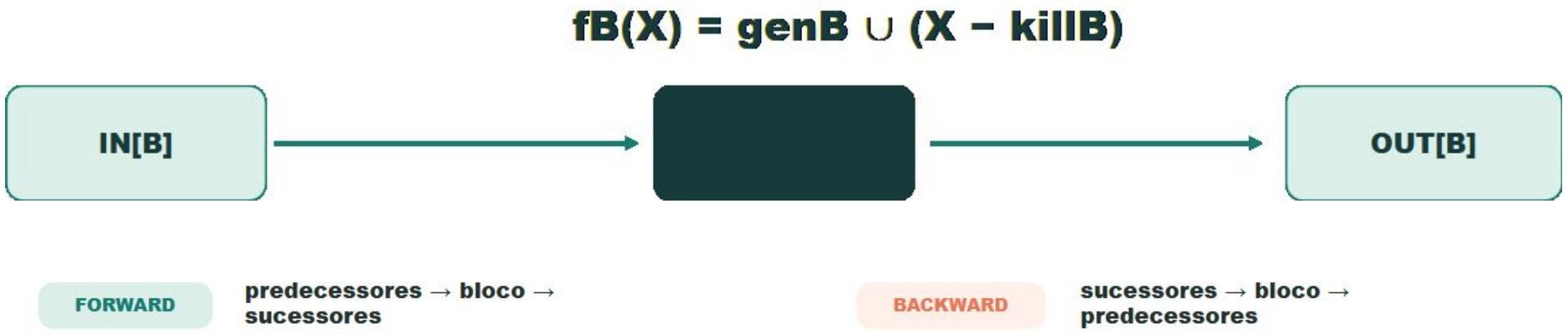
```
t = a[i];
a[j] = 9;
u = a[i];
```

i pode ser igual a j: a escrita invalida a reutilização automática.

Código morto: remover uma definição somente se o valor não for usado em nenhum caminho e a instrução não produzir efeito observável.

Análises forward e backward

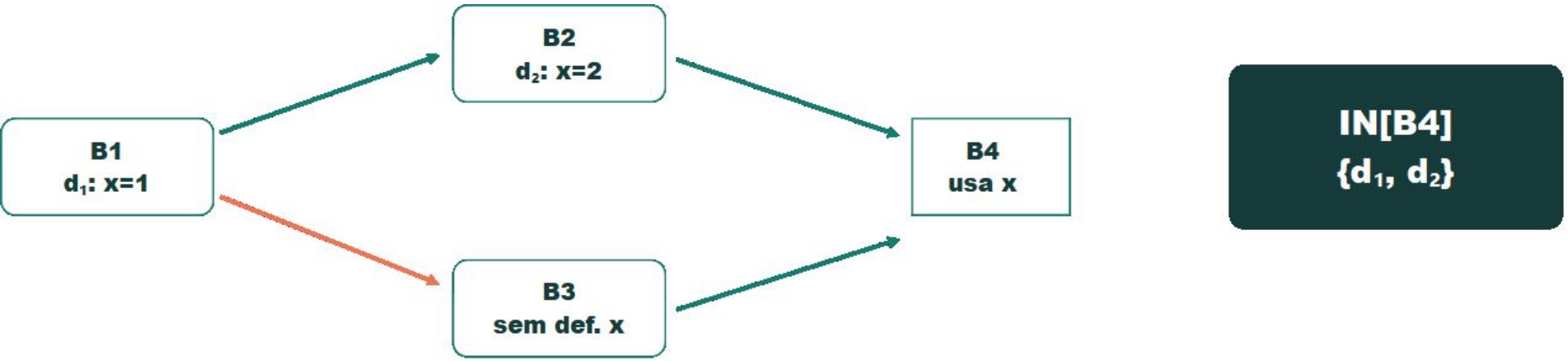
- IN, OUT e função de transferência:
  - gen produz fatos;
  - kill invalida fatos anteriores.



Iterar até estabilizar: ponto fixo. Ciclos normalmente exigem mais de uma passagem.

Análises forward e backward

- Quais definições podem chegar sem redefinição da mesma variável?

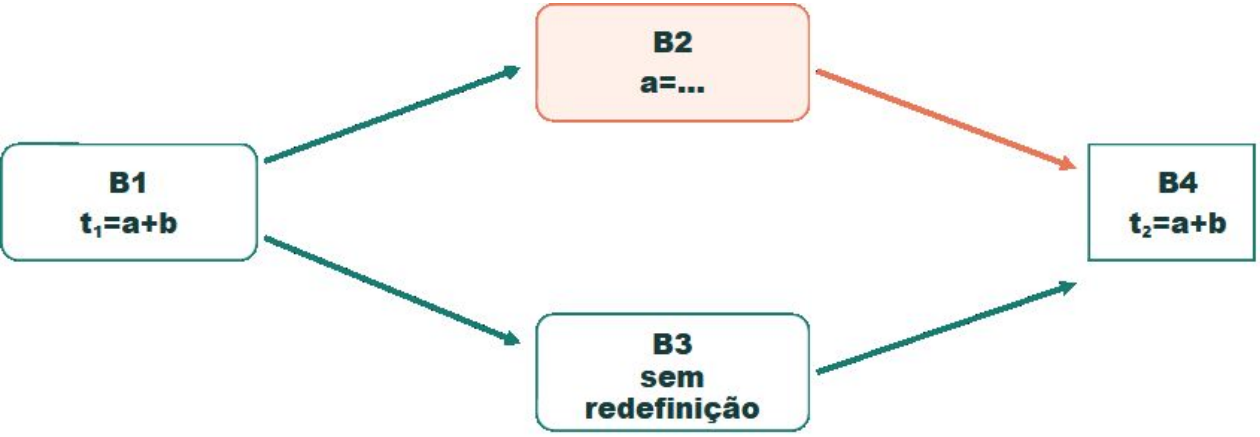


IN[B] = ∪ **OUT[P]** • **OUT[B]** = **genB** ∪ (**IN[B]** − **killB**)

União: basta um predecessor oferecer o caminho. O conjunto expressa alternativas possíveis.

Análises forward e backward

- A expressão precisa ter sido calculada em todos os caminhos, sem mudança dos operandos.



Na junção a+b não está disponível

IN[B] = ∩ OUT[P] • **OUT[B] = e_genB ∪ (IN[B] − e_killB)**

Interseção: todos os predecessores precisam preservar a expressão.

Análises forward e backward

- A variável x está “viva” se seu valor corrente pode ser usado antes de receber nova definição.



B1: use={b,c} def={a}

B2: use={a} def=∅ → a viva na entrada

OUT[B]=∪ IN[S] • **IN[B]=useB ∪ (OUT[B]−defB)**

Análises forward e backward

- Outro mecanismo é a propagação global de constantes.
- Só é possível substituir por constante se o mesmo valor estiver provado em todos os caminhos.

UNDEF sem definição observada

c constante conhecida

NAC não constante / valores divergem

Ramo 1: x=5 • Ramo 2: x=5

→ junção: x↦5; y=x+2 pode virar 7

Ramo 1: x=5 • Ramo 2: x=8

→ junção: NAC; não há literal único

Valores divergentes significam “não sei que constante usar” — não escolha um dos ramos.

Análises forward e backward

- Por fim, percebemos que cada otimização precisa do fato certo.
- Direção e operador de junção respondem à pergunta semântica da análise.

Análise	Direção	Meet	Pergunta	Uso
Reaching definitions	Forward	União	Quais atribuições podem chegar?	Propagação
Available expressions	Forward	Interseção	Calculada em todo caminho?	Eliminar recomputação
Live variables	Backward	União	Quais valores ainda serão usados?	Código morto
Constantes globais	Forward	Junção abstrata	Mesmo valor em todo caminho?	Substituir literal

Fato inconclusivo? Preserve o programa; não otimize com base em um caminho só.

Análises forward e backward

- Perguntas que protegem a otimização:

- Que fato a transformação precisa?
- Vale em todos os caminhos relevantes?
- Tipos, efeitos e falhas permanecem observáveis?